Lanzamiento de dados Android

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Layout principal: ConstraintLayout coloca cada element
o
     mediante "restricciones" (anclajes) respecto a otros e
lementos o al padre -->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:an
droid="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <!-- match_parent: ocupa toda la pantalla (ancho y alt
o) -->
    <!-- tools:context indica qué Activity usa este layout
(solo para el editor) -->

    <!-- ===== PRIMER DADO ===== -->
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="120dp"
        android:layout_height="150dp"
        android:layout_marginEnd="8dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/imageView2"
        app:layout_constraintHorizontal_chainStyle="packed"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:srcCompat="@drawable/dice_1" />
    <!-- id: nombre con el que lo buscamos desde Kotlin (R.
id.imageView) -->
    <!-- 120dp x 150dp: tamaño fijo; dp se adapta a la dens
idad de cada pantalla -->
```

```
    <!-- marginEnd 8dp: separación con el segundo dado -->
    <!-- Top y Bottom al padre: lo centran en vertical -->
    <!-- Start al padre y End al segundo dado: forma una "c
adena" con el otro dado -->
    <!-- chainStyle packed: junta los dos dados y centra el
grupo en horizontal -->
    <!-- tools:srcCompat: imagen solo para la vista previa,
no aparece en el móvil -->

    <!-- ===== SEGUNDO DADO ===== -->
    <ImageView
        android:id="@+id/imageView2"
        android:layout_width="120dp"
        android:layout_height="150dp"
        android:layout_marginStart="8dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/imageView"
        app:layout_constraintTop_toTopOf="parent"
        tools:srcCompat="@drawable/dice_2" />
    <!-- Start anclado al final del primer dado y End al pa
dre: cierra la cadena -->
    <!-- marginStart 8dp: la otra mitad de la separación en
tre dados -->

    <!-- ===== BOTÓN ===== -->
    <Button
        android:id="@+id/btnRoll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Roll"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/imageView"
/>
    <!-- wrap_content: el botón mide lo justo para su texto
-->
```

```
    <!-- Start y End al padre: centrado en horizontal -->
    <!-- Top al fondo del primer dado + 16dp: queda justo d
ebajo de los dados -->

</androidx.constraintlayout.widget.ConstraintLayout>
```

Este es el main_activity

```
package com.example.myapplication

// Importamos las clases de Android que vamos a usar
import android.os.Bundle
import android.util.Log                          // Para escri
bir mensajes en el Logcat
import android.widget.Button                     // El botón
import android.widget.ImageView                  // Las imágen
es de los dados
import androidx.activity.enableEdgeToEdge        // Para que l
a app use toda la pantalla
import androidx.appcompat.app.AppCompatActivity

// MainActivity es la pantalla principal de la app.
// Hereda de AppCompatActivity, que le da todo el comportam
iento de una Activity.
class MainActivity : AppCompatActivity() {

    // companion object: lo que hay dentro pertenece a la c
lase, no a cada objeto
    // (equivale a "static" en Java). Aquí guardamos el tag
para el Logcat.
    companion object {
        const val CICLO_VIDA = "CicloVida"
    }

    // onCreate: se ejecuta UNA vez, cuando se crea la pant
alla.
```

```
    // Aquí se prepara todo: se carga el diseño y se config
uran los botones.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)      // Llama al
onCreate del padre (obligatorio)
        enableEdgeToEdge()                      // La app o
cupa la pantalla de borde a borde
        setContentView(R.layout.activity_main)  // Carga el
diseño de activity_main.xml
        Log.d(CICLO_VIDA, "onCreate")           // Mensaje
en el Logcat para ver el ciclo de vida

        // Buscamos en el diseño los elementos por su id y
los guardamos en variables
        val btnRoll: Button = findViewById(R.id.btnRoll)
        val diceImage1: ImageView = findViewById(R.id.image
View)
        val diceImage2: ImageView = findViewById(R.id.image
View2)

        // Qué pasa al pulsar el botón: se tiran los dos da
dos
        btnRoll.setOnClickListener {
            rollDice(diceImage1)
            rollDice(diceImage2)
        }

        // Tirada inicial para que al abrir la app no salga
la pantalla vacía
        rollDice(diceImage1)
        rollDice(diceImage2)
    }

    // Tira un dado y muestra el resultado en el ImageView
que le pasemos.
    // Al recibir el ImageView como parámetro, sirve para c
ualquier dado
    // y no hace falta repetir la función para cada uno.
```

```
    private fun rollDice(diceImage: ImageView) {
        // Creamos un dado de 6 caras y lo tiramos: devuelv
e un número del 1 al 6
        val diceRoll = Dice(numSides = 6).roll()

        // Descripción para accesibilidad: los lectores de
pantalla dirán el número
        diceImage.contentDescription = diceRoll.toString()

        // Según el número que ha salido, ponemos la imagen
correspondiente
        when (diceRoll) {
            1 -> diceImage.setImageResource(R.drawable.dice
_1)
            2 -> diceImage.setImageResource(R.drawable.dice
_2)
            3 -> diceImage.setImageResource(R.drawable.dice
_3)
            4 -> diceImage.setImageResource(R.drawable.dice
_4)
            5 -> diceImage.setImageResource(R.drawable.dice
_5)
            6 -> diceImage.setImageResource(R.drawable.dice
_6)
        }
    }

    // ===== MÉTODOS DEL CICLO DE VIDA =====
    // Android los llama automáticamente según lo que pasa
con la app.
    // Cada uno escribe en el Logcat para que veas en qué o
rden se ejecutan.

    // onStart: la pantalla se va a hacer visible
    override fun onStart() {
        super.onStart()
        Log.d(CICLO_VIDA, "onStart")
    }
```

```
    // onResume: la pantalla está visible y el usuario ya p
uede interactuar
    override fun onResume() {
        super.onResume()
        Log.d(CICLO_VIDA, "onResume")
    }

    // onPause: la app pierde el foco (ej.: aparece un diál
ogo o sales de ella)
    override fun onPause() {
        super.onPause()
        Log.d(CICLO_VIDA, "onPause")
    }

    // onStop: la pantalla ya no se ve (ej.: has vuelto al
menú del móvil)
    override fun onStop() {
        super.onStop()
        Log.d(CICLO_VIDA, "onStop")
    }

    // onDestroy: la Activity se destruye (ej.: cierras la
app o giras la pantalla)
    override fun onDestroy() {
        super.onDestroy()
        Log.d(CICLO_VIDA, "onDestroy")
    }
}
```

Este es el dice kt

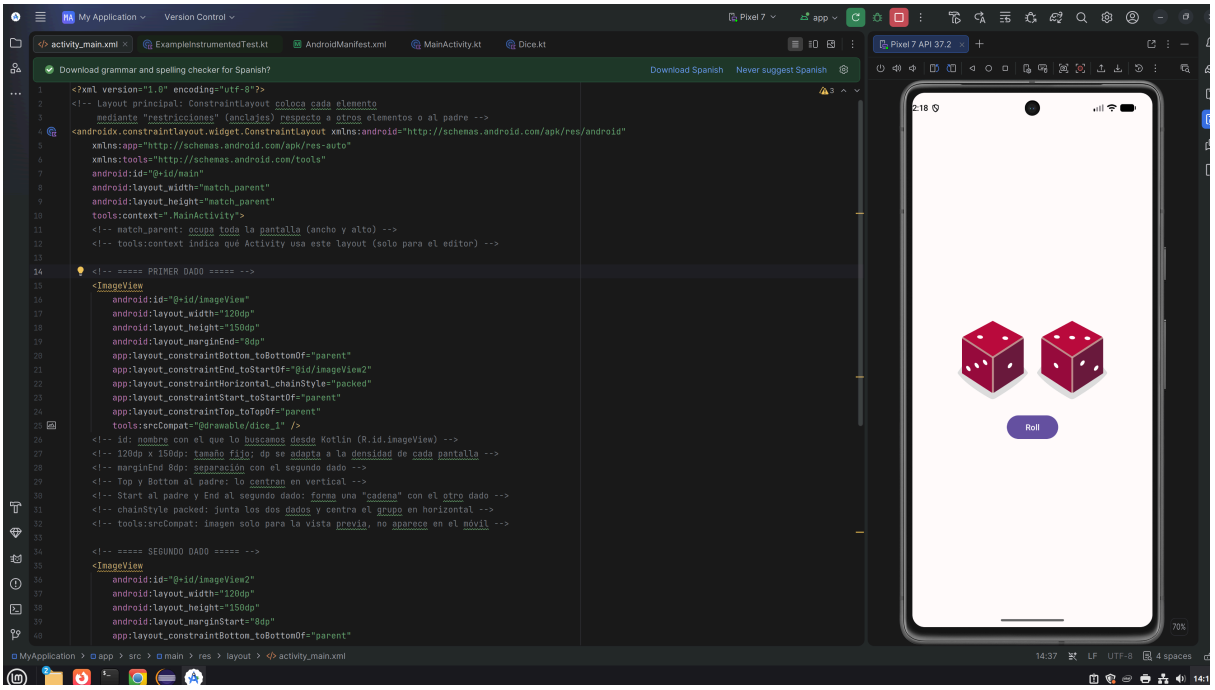
```
package com.example.myapplication

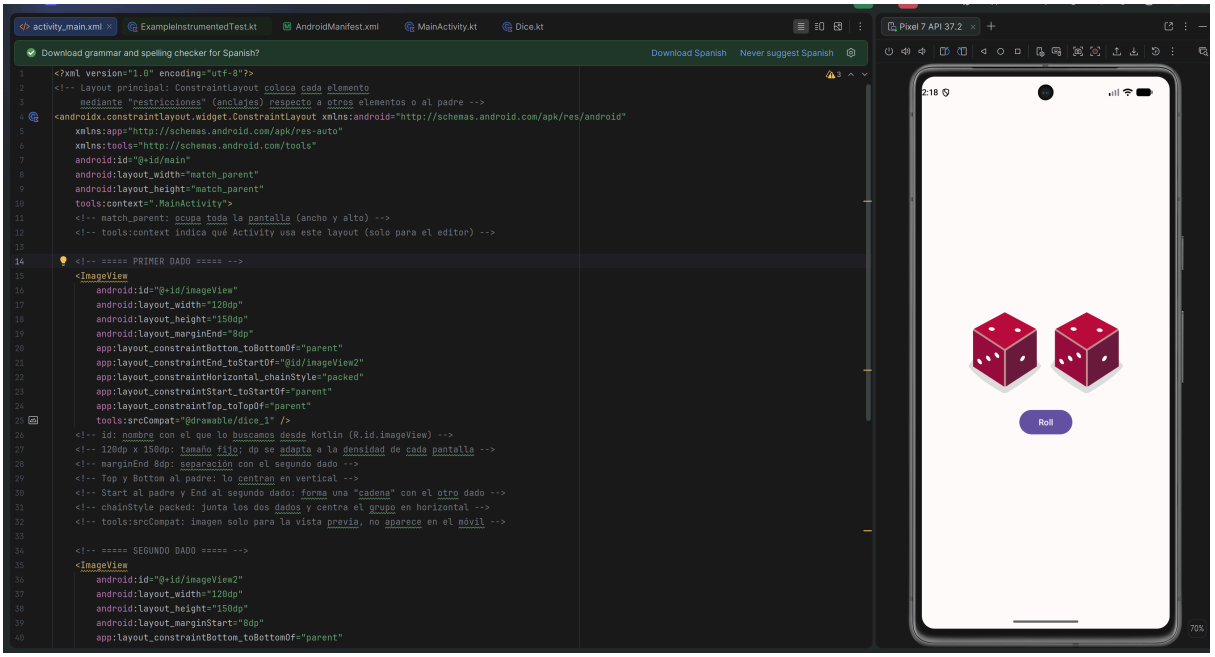
import android.view.WindowInsets

class Dice(val numSides: Int) {
```

```
    fun roll(): Int {
        return (1..numSides).random()
    }
}
```

Estas son las imagenes




activity_main.xml
ExampleInstrumentedTest.kt
AndroidManifest.xml
MainActivity.kt
Dice.kt
Download grammar and spelling checker for Spanish?
Download Spanish
Never suggest Spanish
<?xml version="1.0" encoding="utf-8"?>
<!-- Layout principal: ConstraintLayout coloca cada elemento
mediante "restricciones" (anclajes) respecto a otros elementos o al padre -->
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<!-- match_parent: ocupa toda la pantalla (ancho y alto) -->
<!-- tools:context indica qué Activity usa este layout (solo para el editor) -->
<!-- ===== PRIMER DADO ===== -->
<ImageView
android:id="@+id/imageView"
android:layout_width="120dp"
android:layout_height="150dp"
android:layout_marginEnd="8dp"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintEnd_toStartOf="@id/imageView2"
app:layout_constraintHorizontal_chainStyle="packed"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent"
tools:srcCompat="@drawable/dice_1" />
<!-- id: nombre con el que lo buscamos desde Kotlin (R.id.imageView) -->
<!-- 120dp x 150dp: tamaño fijo; dp se adapta a la densidad de cada pantalla -->
<!-- marginEnd 8dp: separación con el segundo dado -->
<!-- Top y Bottom al padre: lo centran en vertical -->
<!-- Start al padre y End al segundo dado: forma una "cadena" con el otro dado -->
<!-- chainStyle packed: junta los dos dados y centra el grupo en horizontal -->
<!-- tools:srcCompat: imagen solo para la vista previa, no aparece en el móvil -->
<!-- ===== SEGUNDO DADO ===== -->
<ImageView
android:id="@+id/imageView2"
android:layout_width="120dp"
android:layout_height="150dp"
android:layout_marginStart="8dp"
app:layout_constraintBottom_toBottomOf="parent"
Pixel 7 API 37.2
2:18
Roll
70%